Distributed Deep Learning Training on GPU/TPU Clusters: A Critical Literature Review

Course: High Performance Computing | **ISE-II** | **Academic year:** [2026-27]

Student name: Aditi Mittal

PRN: 23610067

Class: L.Y. B.Tech Final Year CSE

Institute: Walchand College of Engineering

Abstract

Training modern deep neural networks on a single accelerator is constrained by device memory, compute throughput, and the time required to process large datasets. Distributed training addresses these limits by dividing work across GPUs or TPUs, but introduces communication overhead, synchronization costs, pipeline idle time, and difficult placement decisions. This review examines 15 research papers published between 2018 and 2022 on distributed deep-learning training, with older foundational work included because it established pipeline-parallel methods. The papers are grouped into data parallelism and communication, pipeline and hybrid parallelism, memory and optimizer-state sharding, and automatic or fine-grained parallelization. Their techniques, reported platforms, evaluation measures, and limitations are compared. Across the studies, no single strategy dominates: data parallelism is simple and effective when gradients can be communicated efficiently; pipeline parallelism reduces communication and permits large models but is sensitive to stage balance and pipeline bubbles; sharding increases model capacity but can add communication or depend on fast memory tiers; and automatic planners reduce expert effort while relying on cost models and supported model structures. Reported speedups are not directly comparable because model, hardware, baselines, and measures differ. The review concludes that future HPC training systems should optimize time-to-quality and cost or energy alongside throughput, improve reproducibility across hardware, and support robust execution under heterogeneous and failure-prone conditions.

Keywords: high-performance computing, distributed deep learning, GPU clusters, data parallelism, tensor parallelism, pipeline parallelism, ZeRO, scalability

1. Introduction

Deep learning workloads increasingly require more computation and memory than a single accelerator can provide. For a training step, a system must execute forward and backward operations, retain or recompute intermediate activations, update parameters, and communicate the resulting state. Increasing accelerator count can shorten execution time and expand the feasible model size, but only when computation scales faster than communication and synchronization costs. In practice, accelerator memory capacity, interconnect bandwidth and topology, model structure, batch size, and software overhead jointly determine performance.

Distributed training uses several complementary decompositions. **Data parallelism** replicates a model and gives each replica different input samples; gradients or parameter updates must then be synchronized. **Tensor parallelism** partitions operations within layers, typically exchanging intermediate values frequently and therefore benefiting from fast intra-node links. **Pipeline parallelism** assigns groups of layers to different devices and overlaps microbatches as they move through stages. **Optimizer-state or parameter sharding** distributes training state that would otherwise be redundantly replicated. Large runs commonly compose these approaches rather than use only one.

This topic matters to HPC because distributed training is a workload in which high arithmetic intensity can be undermined by communication, memory traffic, and load imbalance. System design therefore requires both algorithmic and architectural reasoning: the best partition depends on a cluster's topology and memory hierarchy, not only the number of accelerators. The papers reviewed here demonstrate that increasing model scale is not itself proof of efficient scaling. A useful comparison must distinguish model capacity, throughput, time-to-target quality, scaling efficiency, memory consumption, and infrastructure cost.

This report asks three questions: (1) how do current distributed-training systems divide computation and state; (2) what trade-offs do their evaluations reveal; and (3) which limitations recur across the literature? The review synthesizes 15 papers and does not claim to reproduce their experiments.

2. Review Objectives and Method

2.1 Objectives

1. Summarize the principal parallelization and memory-management approaches used for large-model training.
2. Compare the papers' methods, platforms, metrics, reported findings, and stated or inferable limitations.
3. Explain how communication, memory capacity, synchronization, and scheduling affect performance and scalability.
4. Identify research gaps and propose practical directions for future HPC training systems.

2.2 Paper-selection method

The paper set was assembled through targeted searches of arXiv records, USENIX proceedings, and conference or proceedings pages for distributed training, pipeline parallelism, memory sharding, and large-model training. Search terms included “distributed deep learning training,” “pipeline parallelism,” “optimizer state sharding,” “automatic model parallelism,” and “large-scale language model training.” The main publication window is 2019–2022, with GPipe and PipeDream retained as foundational studies. The selected papers are primary research contributions with identifiable methods and systems evaluations. Abstracts and publisher or conference records were used to confirm bibliographic details and headline findings; detailed hardware and measurement particulars should be checked against each full paper before final submission. This is a focused narrative review, not a systematic review: it does not claim exhaustive database coverage, formal quality scoring, or meta-analysis.

3. Literature Review

3.0 Summary Table of Reviewed Papers

Sr.	Authors / Year	Title (abbreviated)	Problem Addressed	Technique	Platform / Tools	Key Results	Limitations
1	Huang et al., 2019	GPipe	Training models too large for a single device	Synchronous pipeline parallelism with microbatches	TPU v3, TensorFlow	Trained 6B-param Transformer; feasible large-model training	Pipeline bubble; high activation memory
2	Harlap et al., 2019	PipeDream	Communication bottleneck and throughput limits in data parallelism	Asynchronous pipeline with automatic layer partitioning	GPU cluster, PyTorch	Up to 5× faster time-to-accuracy; up to 95% less communication	Asynchronous updates complicate convergence; versioned weights add memory
3	Shoeybi et al., 2019	Megatron-LM	Scaling transformer models beyond single-GPU memory	Tensor (model) parallelism within layers	NVIDIA DGX-2, 512 GPUs	8.3B-param model; 76% scaling efficiency	Communication-intensive; limited to fast intra-node links
4	Rajbhandari et al., 2020	ZeRO	Redundant optimizer state / gradient storage in data parallelism	Partition optimizer states, gradients, and parameters across workers	NVIDIA GPU cluster, DeepSpeed	>100B-param models on 400 GPUs; 15 petaflops	Communication overhead for parameter gathering; interconnect dependent
5	Fan et al., 2021	DAPPLE	Stage imbalance and memory use in synchronous pipelines	Combined synchronous data and pipeline parallelism with automatic planner	NVIDIA GPU cluster	1.6× throughput over GPipe; 12% lower memory	Planner accuracy depends on hardware cost model
6	Li et al., 2020	PyTorch DDP	Communication overhead in synchronous data parallelism	Gradient bucketing with backward-communication overlap	NVIDIA GPU cluster, PyTorch	Near-linear scaling to 256 GPUs	Scaling is workload- and configuration-specific
7	Narayanan et al., 2021	Megatron-LM (SC'21)	Scaling to trillion-parameter models on large GPU clusters	Tensor + pipeline + data parallelism (3D hybrid)	3,072 NVIDIA A100 GPUs	1T-param training; 502 petaflops; 52% of peak FLOPs	Requires large, specialized cluster; 48% FLOPs still idle

8	Rajbhandari et al., 2021	ZeRO-Infinity	GPU memory wall limiting trainable model size	Heterogeneous memory offload (GPU + CPU + NVMe) via DeepSpeed	512 NVIDIA V100 GPUs, NVMe	>25 petaflops; 1T-param fine-tuning on single DGX-2	Offload bandwidth sensitive; storage I/O can be bottleneck
9	Athlur et al., 2021	Varuna	High cost of large-model training on dedicated clusters	Pipeline parallelism on low-priority cloud VMs with automatic configuration	AWS spot VMs, commodity networking	200B-param model training; up to 18× over model-parallel baseline	Results depend on spot pricing, interruptions, and fault handling
10	Li & Hoefler, 2021	Chimera	Pipeline bubble overhead in synchronous pipelines	Bidirectional pipelines to reduce idle stages	NVIDIA GPU cluster	1.16×–2.34× over single-direction pipeline baselines	Scheduling and placement complexity; requires suitable pipeline structure
11	Narayanan et al., 2021	PipeDream-2BW	Activation and weight memory overhead in pipeline parallelism	Weight double buffering + gradient coalescing + automatic partitioning	NVIDIA GPU cluster	Up to 20× acceleration vs. baselines for GPT/BERT training	Large speedup claim is baseline-specific; details matter for reproduction
12	Zheng et al., 2022	Alpa	Manual effort in designing parallelism strategies	Hierarchical compiler passes for inter- and intra-operator parallelism	NVIDIA GPU cluster, JAX	Matches or beats hand-tuned plans; supports heterogeneous hardware	Cost-model accuracy; limited support for dynamic/irregular models
13	Korthikanti et al., 2022	Sequence Parallelism	High activation memory from full recomputation in Transformers	Sequence parallelism + selective activation recomputation	2,240 NVIDIA A100 GPUs	54.2% MFU vs. 42.1%; 5× activation memory reduction	Specific to large Transformer workloads on large clusters
14	Li et al., 2021	TeraPipe	Coarse pipeline granularity for autoregressive models	Token-level pipeline parallelism with dynamic programming schedule	48× AWS p3.16xlarge, PyTorch	Up to 5× speedup for 175B GPT-3 vs. model-parallel baseline	Requires autoregressive structure; not general to all model types

15	Lepikhin et al., 2021	GShard	Scaling beyond dense model capacity limits	Sparse Mixture-of-Experts + compiler-based automatic sharding	2,048 TPU v3 accelerators	600B+ param multilingual model in 4 days	TPU-specific infrastructure; routing imbalance in sparse models

3.1 Data-parallel training and communication efficiency

Synchronous data parallelism is attractive because each worker runs the same model on different data. Its simplicity does not eliminate the communication bottleneck: gradients must be aggregated, and each worker can wait for communication or for a slower peer. Li et al. describe PyTorch Distributed Data Parallel (DDP), where gradient bucketing and overlapping communication with backward computation improve utilization. Their evaluation reports near-linear scaling to 256 GPUs when configured appropriately [6]. This is useful evidence that a mature framework can make a familiar strategy scale, but "near-linear" is conditional on the tested models, cluster, and configuration; it is not a general guarantee for arbitrary network or batch sizes.

The Megatron-LM work by Shoeybi et al. provides a complementary model-parallel approach. It partitions transformer operations across GPUs, enabling models too large for one device; the paper reports training an 8.3-billion-parameter model with 512 GPUs and 76% scaling efficiency relative to its stated single-GPU baseline [3]. Narayanan et al. later combine tensor, pipeline, and data parallelism in a production-oriented configuration. Their SC 2021 paper reports a 1-trillion-parameter training iteration at 502 petaflops on 3,072 GPUs, with per-GPU throughput at 52% of theoretical peak [7]. Together, these results show the importance of matching parallelism to model size and interconnect. They are not directly comparable: the systems use different model sizes, cluster generations, baselines, and definitions of efficiency.

The central HPC lesson is that aggregation overhead depends on more than total bytes. Collective algorithms, message size, topology, overlap opportunities, and compute per synchronized step determine whether adding workers helps. Data parallelism can remain effective when each worker performs enough computation per update and the network supports collectives; as models or worker counts grow, hybrid decompositions can be necessary.

3.2 Pipeline and hybrid parallelism

GPipe introduced a broadly applicable synchronous pipeline design: split a batch into microbatches, assign consecutive layer groups to devices, and pipeline forward and backward computation. The work demonstrates large image and multilingual translation models, including a 6-billion-parameter Transformer [1]. Its key benefit is model capacity and concurrency; its key weakness is the pipeline "bubble," when stages wait while the pipeline fills or drains. Stage imbalance and activation storage can further reduce utilization.

PipeDream explores asynchronous pipeline execution and automatically partitions layers to balance work and communication. Its evaluation reports up to 5x faster time-to-accuracy than data-parallel

training in its tested workloads and up to 95% lower communication for large DNNs [2]. The result is promising but should be interpreted with its consistency mechanism in mind: versioned weights and asynchronous updates complicate the relationship between throughput and convergence. A speedup in time-to-target accuracy is more informative than raw iteration throughput, yet it still depends on the models and targets used.

DAPPLE combines synchronous data and pipeline parallelism, with a planner for partition and placement and a schedule aimed at reducing memory use without adding recomputation [5]. In its evaluation, the planner improves over the PipeDream planner in the tested synchronous scenarios, while its runtime reports 1.6x throughput over GPipe and 12% lower memory consumption. These are within-study comparisons, not universal rankings. DAPPLE's synchronous semantics make convergence reasoning clearer than asynchronous pipelines, but a planner still needs accurate hardware and workload estimates.

PipeDream-2BW targets pipeline memory overhead by combining weight double buffering with gradient coalescing and automatic partitioning. The paper reports up to 20x acceleration for tested GPT and BERT training relative to its baselines while retaining similar final accuracy [11]. The unusually large maximum emphasizes why baseline and setup details matter: the value should not be read as a guaranteed improvement over any modern framework. Chimera uses bidirectional pipelines to reduce idle bubbles while preserving synchronous training semantics; on its evaluated Transformer workloads, it reports 1.16x–2.34x throughput over selected pipeline baselines [10]. The approach improves pipeline balance but adds scheduling and placement complexity and assumes a suitable pipeline structure.

TeraPipe explores a finer unit of pipeline execution, splitting work at token level within autoregressive Transformer sequences. It uses dynamic programming to choose an execution schedule for a given model and cluster. The authors report up to 5x speedup for a 175-billion-parameter GPT-3 model on an AWS cluster of 48 p3.16xlarge instances compared with selected model-parallel approaches [14]. Its advantage relies on autoregressive structure and does not automatically transfer to models without a sequence dependency that permits this schedule.

Varuna focuses on training massive models using more economical, lower-priority cloud VMs and commodity networking. It combines pipelining, automatic configuration, and mechanisms designed for variable resources. The authors report training a 200-billion-parameter model on spot VMs and up to 18x improvement in end-to-end training time over selected model-parallel alternatives [9]. This shifts the optimization target from peak accelerator throughput to cost and resource availability. The trade-off is that results depend on cloud pricing, spot interruption behavior, and workload-specific fault handling, making comparisons with dedicated supercomputers especially difficult.

3.3 Memory capacity, optimizer states, and recomputation

Model parameters are only one part of training memory. Gradients, optimizer states, and activations can consume several times the parameter storage. ZeRO partitions optimizer state and then gradients and parameters across data-parallel workers to remove redundant copies. The paper reports training models over 100 billion parameters with 400 GPUs and 15 petaflops, and describes super-linear

speedup in its evaluation [4]. The key insight is that sharding changes the memory-per-worker constraint and can enable larger models without requiring developers to rewrite a model into layer partitions. However, communication and parameter-gathering costs remain, and the benefits depend on the amount of state that can be sharded efficiently.

ZeRO-Infinity extends this direction across a heterogeneous memory hierarchy, using GPU, CPU, and NVMe storage to exceed GPU-memory limits. Its evaluation reports over 25 petaflops on 512 NVIDIA V100 GPUs and the ability to fine-tune a trillion-parameter model on a single DGX-2 [8]. This is a notable capacity result, but offloading is not free: practical performance depends on transfer volume, storage bandwidth, prefetching, and whether transfers can overlap computation. The study illustrates a broader HPC principle: expanding the memory hierarchy can increase feasible problem size while shifting the bottleneck to data movement.

Korthikanti et al. examine the cost of activation recomputation in Transformer training. Sequence parallelism and selective recomputation reduce activation memory while avoiding much of the redundant computation of full recomputation. On a 530-billion-parameter model using 2,240 NVIDIA A100 GPUs, the paper reports 54.2% model FLOPs utilization, compared with 42.1% under the recomputation baseline, and a 5x reduction in activation memory [13]. These results illustrate that memory optimization should be evaluated jointly with execution time: saving memory by recomputing can reduce throughput, while selective strategies may improve both. The evaluation remains tied to very large Transformer workloads and a large accelerator cluster.

3.4 Automatic sharding and conditional computation

GShard combines lightweight model annotations with compiler-based automatic sharding and sparse Mixture-of-Experts computation. It reports a multilingual translation model with more than 600 billion parameters trained on 2,048 TPU v3 accelerators over four days [15]. The study shows the capacity unlocked by conditional computation and compiler support, but its scale is enabled by specialized TPU infrastructure and a particular model family. Sparse models also raise additional issues, including routing imbalance and communication patterns among experts.

Alpa addresses the burden of manually composing parallelism strategies. It treats inter-operator and intra-operator parallelism as hierarchical planning problems and uses compiler passes and a runtime to generate execution plans [12]. The authors report plans that match or outperform hand-tuned systems on tested models and generalize to heterogeneous architectures without hand-designed plans. Automation can lower the expertise barrier and make search over strategies practical; nevertheless, search-space coverage, cost-model accuracy, compile time, and support for dynamic or irregular models remain important constraints. An automatically selected plan is only as robust as its performance model and hardware assumptions.

4. Methodologies Used in the Reviewed Papers

The papers use four recurring methodological families:

- **Data parallelism:** replicate model computation and synchronize gradients or updates. DDP is the clearest example [6]. Its effectiveness depends on collective communication, overlap, batch size, and compute-to-communication ratio.
- **Tensor/model parallelism:** split individual operators or layers across accelerators. Megatron-LM is a prominent example [3], often paired with pipeline and data parallelism at larger scale [7]. Frequent intra-layer communication makes interconnect topology central.
- **Pipeline and hybrid parallelism:** partition layers into stages, schedule microbatches, and optionally combine pipeline stages with data parallel replicas. GPipe, PipeDream, DAPPLE, PipeDream-2BW, Chimera, TeraPipe, and Varuna represent different synchronization, scheduling, and placement choices [1], [2], [5], [9]–[11], [14].
- **State sharding and compiler-based partitioning:** distribute parameters, gradients, optimizer states, activations, or operators to relieve memory limits. ZeRO and ZeRO-Infinity focus on state partitioning and offload [4], [8]; GShard and Alpa emphasize automatic compiler-supported partitioning [12], [15].

Evaluation typically uses throughput (samples, tokens, or FLOPs per second), scaling efficiency, time-to-accuracy, memory footprint, model size, or some combination. Platforms include NVIDIA GPU clusters, cloud GPU instances, and TPU v3 pods. Training tasks include image classification, machine translation, and language-model workloads. The review did not find a common benchmark, fixed cluster, and uniform reporting protocol across all papers. Energy consumption, total monetary cost, failure/recovery overhead, and variance across repeated runs are less consistently reported than throughput and scale. Therefore, numerical results should be treated as within-paper evidence rather than pooled into a cross-paper leaderboard.

5. Comparative and Critical Analysis

5.1 What the evidence supports

First, parallelism is a resource-allocation decision, not a one-dimensional speed knob. Data parallelism offers a straightforward path when communication is affordable, but large model states and gradient traffic limit its use. Tensor parallelism reduces per-device operator size but raises communication sensitivity. Pipeline parallelism keeps a large model distributed while reducing some cross-worker communication, but introduces bubbles, stage imbalance, and activation-lifetime constraints. Hybrid systems such as DAPPLE and Megatron-LM attempt to place each form where its costs are lowest [5], [7].

Second, memory capacity can be as decisive as arithmetic throughput. ZeRO-style sharding, heterogeneous offload, and selective activation recomputation each target different memory components [4], [8], [13]. Their comparison should include both peak memory and the cost of moving or regenerating data. A technique that makes a model fit is valuable even if it does not maximize raw throughput, but capacity claims should not be confused with cost-effective training.

Third, reported “speedup” has several meanings. PipeDream reports time-to-accuracy, while other papers report throughput, training time, or FLOPs utilization. A faster step may require more steps to reach the same validation quality; a higher maximum model size may be achieved with expensive hardware; and scaling efficiency depends on the chosen single-device or smaller-cluster baseline. As a result, the reported 1.6x, 5x, 18x, or 20x figures in different studies cannot be ranked directly [2], [5], [9], [11], [14].

Finally, automation is increasingly important. Manual decisions about tensor dimensions, stage boundaries, replica counts, and topology are difficult to make reliably. Alpa and GShard show that compiler support can help, while Varuna and DAPPLE show value in automatic configuration and planning [5], [9], [12], [15]. However, automation does not remove the need for careful objective functions and validation; a planner optimized only for steady-state throughput may ignore cost, convergence, or recovery.

5.2 Strengths and weaknesses by approach

Approach	Main strength	Main weakness / condition
Data parallelism	Simple programming model; mature framework support; strong scaling when collectives are efficient	Gradient communication and synchronization can dominate; memory is replicated without sharding
Tensor parallelism	Splits large operators and model state; effective within fast-connected accelerator groups	Frequent communication makes topology and bandwidth critical; more complex implementation
Pipeline parallelism	Supports models larger than a device; can reduce communication relative to full replication	Pipeline bubbles, stage imbalance, and activation memory reduce utilization
Optimizer/state sharding	Removes replicated training state and increases feasible model size	Parameter gathering/offload adds traffic; performance depends on interconnect and memory tier

Compiler/planner automation	Reduces manual strategy effort and explores more configurations	Search and cost-model errors; portability across model types and hardware is not guaranteed

5.3 Trends over time

The literature progresses from demonstrating pipeline execution and model partitioning to combining multiple parallel dimensions at thousands of accelerators. It also shifts from treating GPU memory as a fixed boundary to partitioning optimizer states, exploiting CPU/NVMe tiers, and selectively recomputing activations. Another trend is moving expertise into planners and compilers: researchers increasingly seek systems that select a strategy from model and cluster characteristics instead of requiring every user to hand-design one. The optimization objective is also broadening: work such as Varuna emphasizes economical resources and end-to-end time rather than only peak FLOPs [9].

6. Research Challenges and Future Scope

5. **Comparable evaluation.** Establish reproducible benchmark suites that report model quality, tokens or samples per second, time-to-target quality, memory, energy, and cost under clearly stated topology and software versions. Report scaling curves, not only the best point.
6. **Energy and cost awareness.** Most studies emphasize throughput or maximum model scale. Future planners should include energy per useful training token, accelerator utilization, cloud pricing, and embodied or facility-level energy where measurable.
7. **Heterogeneous and unreliable resources.** Systems should adapt to different accelerator types, link speeds, memory capacities, and interruptions. Checkpointing, elastic membership, and recovery time should be evaluated as first-class performance metrics.
8. **Communication-aware scheduling.** More work is needed on overlap, topology-aware placement, collective scheduling, and balancing communication against computation across multiple parallelism dimensions, especially for sparse and expert-routed models.
9. **Convergence-aware optimization.** Scheduling and precision changes must preserve target model quality. Evaluations should consistently report validation curves and final quality, rather than throughput alone.
10. **Usability and portability.** Automatic systems should expose their selected plans, explain constraints, and make performance failures diagnosable. Compiler and runtime abstractions need to support changing model architectures without sacrificing efficient hardware-specific execution.
11. **Reproducible reporting.** Publish scripts, configurations, hardware topology, software versions, and enough raw measurements to independently reproduce the main scaling and memory results.

Author's observations and suggestions. The most useful next step is not another isolated peak-scale demonstration, but a shared evaluation protocol that measures time-to-quality, cost, and energy on more than one cluster type. A practical HPC system should make hybrid plans automatically, yet also expose the communication and memory trade-offs behind its decisions. Small-scale reproducibility matters: techniques should be testable on a modest cluster before their claims are extrapolated to thousands of accelerators.

7. Conclusion

This review met its objectives by summarizing 15 distributed-training studies, categorizing their methodologies, and comparing the trade-offs that shape their results. Data parallelism remains a strong baseline when communication can be overlapped; tensor and pipeline parallelism make larger models possible but depend on topology, scheduling, and balanced stages; sharding and selective recomputation manage memory at the cost of additional data movement or implementation complexity; and compiler-driven planning can reduce manual effort while introducing model- and cost-model dependencies. The central conclusion is that there is no universally best parallelization strategy. Effective HPC training requires a workload- and hardware-aware composition of techniques, evaluated by model quality, elapsed time, memory, scalability, energy, and cost together. Future studies should improve cross-system reproducibility and report end-to-end outcomes in addition to peak throughput.

References

[1] Y. Huang et al., “GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. [Online]. Available: https://arxiv.org/abs/1811.06965

[2] A. Harlap et al., “PipeDream: Fast and Efficient Pipeline Parallel DNN Training,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019. [Online]. Available: https://arxiv.org/abs/1806.03377

[3] M. Shoeybi et al., “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” arXiv:1909.08053, 2019. [Online]. Available: https://arxiv.org/abs/1909.08053

[4] S. Rajbhandari et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020. [Online]. Available: https://arxiv.org/abs/1910.02054

[5] S. Fan et al., “DAPPLE: A Pipelined Data Parallel Approach for Training Large Models,” in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021. [Online]. Available: https://arxiv.org/abs/2007.01045

[6] S. Li et al., “PyTorch Distributed: Experiences on Accelerating Data Parallel Training,” *Proceedings of the VLDB Endowment*, 2020. [Online]. Available: https://arxiv.org/abs/2006.15704

[7] D. Narayanan et al., “Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM,” in *Proceedings of SC*, 2021. [Online]. Available: https://arxiv.org/abs/2104.04473

[8] S. Rajbhandari et al., “ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning,” in *Proceedings of SC*, 2021. [Online]. Available: https://arxiv.org/abs/2104.07857

[9] S. Athlur et al., “Varuna: Scalable, Low-cost Training of Massive Deep Learning Models,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, 2021. [Online]. Available: https://arxiv.org/abs/2111.04007

[10] S. Li and T. Hoefler, “Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines,” in *Proceedings of SC*, 2021, doi: 10.1145/3458817.3476145. [Online]. Available: https://arxiv.org/abs/2107.06925

[11] D. Narayanan et al., “Memory-Efficient Pipeline-Parallel DNN Training,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021. [Online]. Available: https://arxiv.org/abs/2006.09503

[12] L. Zheng et al., “Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning,” in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin

[13] V. A. Korthikanti et al., “Reducing Activation Recomputation in Large Transformer Models,” in *Proceedings of SC*, 2022. [Online]. Available: https://arxiv.org/abs/2205.05198

[14] Z. Li et al., “TeraPipe: Token-Level Pipeline Parallelism for Training Large-Scale Language Models,” in *Proceedings of ICML*, 2021. [Online]. Available: https://arxiv.org/abs/2102.07988

[15] D. Lepikhin et al., “GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding,” in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://arxiv.org/abs/2006.16668